Stärken und Schwächen Analyse für die serielle Schnittstelle

Version	Datum	Autor	Änderungsgrund / Bemerkungen
0.0.1	15.10.2026	Stefan Foltrauer	Erstellen des Dokumentes
1.0.0	17.10.2026	Stefan Foltrauer	Erstellen des Literaturverzeichnisses und der Versionsbezeichnung so dass das Dokument veröffentlicht und referenziert werden kann.

Für die Konfigurationsschnittstelle wird eine serielle Schnittstelle notwendig sein.

Um eine gute Entscheidung zu treffen, wurde eine kurze stärken und schwächen Analyse der verschiedenen seriellen Schnittstellen durchgeführt. Die Auflistung der Kriterien mit Bewertung ist in Tabelle 1 ersichtlich. Unterhalb der Tabelle befinden sich jeweils Kommentare zu den Kriterien.

Tabelle 1: Kriterienauflistung mit jeweils einem Punktewert pro Schnittstelle zwischen 1 (schlecht) und 5 (sehr gut)

Nr.	Kriterien	SPI	JTAG	UART
1	Anzahl notwendiger Anschlüsse zum FPGA	4	3	5
2	Aufwand der Implementierung in VHDL	5	1	4
3	Vorhandene VHDL-Beispiele	5	3	4
4	Vertrautheit (Hat der Projektleiter bereits mit der Schnittstelle gearbeitet?)	4	1	3
5	Steuerung des Konfigurationsmasters	4	2	5
	Gesamt:	**22**	**10**	**21**

1) Bei SPI braucht es: MOSI, MISO, CLK (CS aber den kann man hier wohl weglassen). Bei JTAG: TDI, TDO, TCK, TMS (je nach Variantionen gibt es scheinbar auch 2 und 5). Bei UART sind es immer nur Rx und Tx.
2) Der Aufwand ist stark davon abhängig, was man alles im Protokoll möchte. Die folgende Auflistung ist nicht komplett, sondern ist lediglich, was man nach einer kurzen Recherche mit ChatGPT für notwendige Logikbauteile findet. Es wurde als Voraussetzung angenommen, dass kontrolliert werden muss, ob die Übertragung korrekt war.

Tabelle 2: Auflistung der wichtigsten Logikkomponenten je nach serieller Schnittstelle und deren Aufwand zum Implementieren

Logikblock	**SPI**	**JTAG** (Abhängig von der Norm)	**UART**
Kontrolllogik (FSM)	Einfach (Der Tackt wird vom Master vorgegeben)	Komplex (16 TAP-Zustände + Befehlssteuerung)	Mittel (wegen dem erkennen der Start und Endbits)
Schieberegister	1–2 Register	Mehrere Register (IR, DR)	1–2 Register
Zähler	Einfacher Bitzähler	Bitzähler und zusätzliche Steuerlogik (welches Register etc.)	Bitzähler und Baudratenzähler (da keine Clock vom Master)
Taktsynchronisation	Mittel	Mittel	Mittel bis hoch
Fehlererkennung	CRC / Prüfsumme	CRC / Prüfsumme	CRC / Prüfsumme
ACK/NACK-Logik	Einfach	Mittel bis komplex	Einfach
Befehlsdecoder	Einfach	Mittel bis komplex	Einfach

3) Für diesen Punkt wurde eine Internetsuche nach Beispielen gemacht. Die Idee ist, dass man, um den Aufwand zu verkleinern auf bereits vorhandene Beispiele zurückgreifen könnte.
   Es wird keine Auflistung aller Beispiele hier gegeben, da das zu Umfangreich für die Einschätzung wäre.
4) Es wurde noch nie mit JTAG gearbeitet, während SPI und UART bekannt sind.
5) Bei allen braucht man ein Adapter für den Anschluss an den PC oder verwendet das Analog Discovery 2 Kit [1], da man ja von einem ASIC-Design ausgeht und man somit die Schnittstelle über normale Ein- und Ausgänge machen will.

   Die Unterschiedliche Punktzahl kommt zustande, da es wichtig ist, wie einfach Signale vom PC auf den USB übertragen werden können. Das ist wichtig, da noch nicht klar ist, ob der Konfigurationsmaster für den Prototypen über das Analog Discovery 2 laufen wird oder man ihn alternativ mittels eines Programms auf dem PC erstellt und über einen Adapter in das FPGA einspeist. Die Adaptervariante hätte den Vorteil, dass weniger Anschlüsse vom Analog Discovery 2 verwendet werden und man diese stattdessen für das Ein- und Auslesen der FPGA Ein- und Ausgänge verwenden könnte.

   Laut den Erklärungen von ChatGPT wäre das für UART mit Python[2] + PySerial[3] sehr einfach, für SPI mit Python[2]+PyFtdi[4] einfach und für OpenOCD[5] / JTAG-Tools eher Aufwendig.

Literaturverzeichnis:

Die Quellen sind im IEEE-Stil der Ingenieurstudiengänge BFH angegeben.

[1] Digilent, „Analog Discovery 2: 100MS/s USB Oscilloscope, Logic Analyzer and Variable Power Supply,“ [Webseite]. Zugriff: https://digilent.com/shop/analog-discovery-2-100ms-s-usb-oscilloscope-logic-analyzer-and-variable-power-supply/, 17. Okt. 2026.
[2] Microsoft, „Python in Visual Studio Code,“ [Webseite]. Zugriff: https://code.visualstudio.com/docs/languages/python, 17. Okt. 2026.
[3] pySerial, „Welcome to pySerial’s documentation,“ [Webseite]. Zugriff: https://pythonhosted.org/pyserial/, 17. Okt. 2026.
[4] T. É. Blot, „PyFtdi: FTDI device driver written in pure Python,“ [Webseite]. Zugriff: https://github.com/eblot/pyftdi, 17. Okt. 2026.
[5] OpenOCD, „OpenOCD – Open On-Chip Debugger,“ [Webseite]. Zugriff: https://github.com/openocd-org/openocd, 17. Okt. 2026.

Anhang

Im Rahmen der Erstellung dieser Arbeit wurde ChatGPT von OpenAI als unterstützendes Hilfsmittel verwendet. Die Verwendung erfolgte für Fragen allgemeiner Natur sowie zur Unterstützung bei der Erstellung des Literaturverzeichnisses.
Die verwendeten Quellen und die durch das KI-System bereitgestellten Informationen wurden eigenständig geprüft. Die Verantwortung für Inhalt, fachliche Richtigkeit und korrekte Quellenangaben liegt beim Verfasser.